



武汉大学

WUHAN UNIVERSITY

数据结构与程序设计

(Python语言)

DATA STRUCTURE AND PROGRAM DESIGN IN PYTHON

第08讲 文件与异常



Python课程组



2025年02月



CONTENTS

目 录

1 数据存储

2 文件对象

3 文本文件

4 CSV文件

5 对象序列化

6 异常处理

7 实践指导



武汉大学
WUHAN UNIVERSITY

01

数据存储



■ **大数据时代，数据类型复杂繁多，主要包括结构化数据、半结构化数据及非结构化数据等。**

✿ 结构化数据

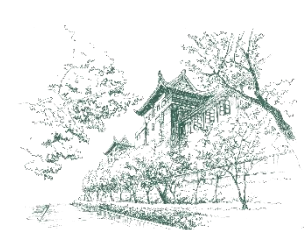
- 根据特定的数据模型组织起来的数据，它明确地定义了数据的结构。
- 结构化数据一般由二维表结构来逻辑表达和实现，严格地遵循数据格式与长度规范，主要通过关系型数据库进行存储和管理。

✿ 非结构化数据

- 数据结构不规则或不完整，没有预定义的数据模型，不方便用二维表来表现的数据，如有格式的文档、各类报表、图形、图像、音频和视频等。

✿ 半结构化数据

- 介于完全结构化和完全非结构化之间的数据，如XML、HTML和JSON等。
- 非结构化和半结构化数据通常用文件系统和非关系数据库存储。



数据存储



■ 数据存储的主要方式包括文件系统存储、关系数据库存储和非关系数据库存储等。

✿ 文件系统存储

➤ 存储数据的常用文件格式有TXT、CSV、JSON和多媒体文件等。

✿ 关系数据库存储

➤ 常见的关系数据库有MySQL、SQLite、Access、Oracle、DB2和SQL Server等。

✿ 非关系数据库存储

➤ 如Redis、HBase、Cassandra、MongoDB、CouchDB、Neo4J和GraphDB等。



武汉大学
WUHAN UNIVERSITY

02

文件对象

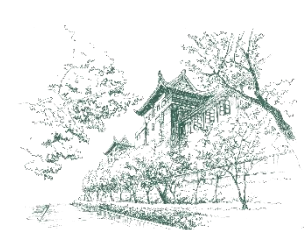


文件分类



■ 文件按组织形式可以分为文本文件和二进制文件两类。

- ✱ 文本文件用于存储文本字符串，默认编码与操作系统相关（如Windows中文环境默认为GBK）。文本文件可以使用字处理软件（如记事本、VS Code等）直接编辑。
- ✱ 二进制文件把对象内容以字节串形式存储，无法直接通过文本编辑器查看。常见的Office文档、图形图像文件、音视频文件、可执行文件和各种数据库文件等都属于二进制文件。



文件操作的一般流程



武汉大学
WUHAN UNIVERSITY

■ 无论是文本文件还是二进制文件，其操作流程基本上是一致的。

- ✱ (1) 打开文件并创建文件对象；
- ✱ (2) 通过文件对象对文件内容进行写入、读取、删除和修改等操作；
- ✱ (3) 关闭文件。



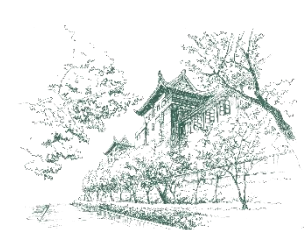
Python内置了文件对象，通过`open()`函数以指定模式打开文件，并创建文件对象。

`open()`函数的一般形式

`open(file, mode='r', encoding=None)`

其中，`file`是要打开或创建的文件名，如果文件不在当前路径下，需指出具体路径；`mode`是打开文件的模式；`encoding`是文件的编码，仅在文本模式下使用。

模 式	说 明
'r'	读取（默认模式，文件需存在）
'w'	写入（覆盖旧内容，文件不存在则创建）
'x'	独占创建（文件存在则报错）
'a'	追加（文件存在时末尾写入，不存在则创建）
'b'	二进制模式（需与 r/w/a/x 组合使用，如 rb）
't'	文本模式（默认，可与 r/w/a/x 组合使用）
'+'	读写模式（需与 r/w/a 组合使用，如 r+或 w+）



■ Python内置了文件对象，通过`open()`函数以指定模式打开文件，并创建文件对象。

■ `open()`函数的一般形式

`open(file, mode='r', encoding=None)`

```
file1 = open("input.txt") # 以读模式打开当前路径下的input.txt文件  
file2 = open(r"d:\temp\output.txt", 'w') # 以写模式打开d:\temp路径下的output.txt文件
```

- 如果`open()`函数执行正常，则返回1个文件对象，通过该文件对象可以对文件进行读写操作。如果指定文件不存在、访问权限不够、磁盘空间不足或其他原因导致创建文件对象失败则抛出异常。
- 注意，在Windows系统中，路径名中往往出现反斜杠（\），为了避免被当作转义字符，一般用原始字符串来表示路径。



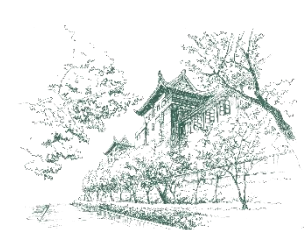
■ 文件对象的常用属性

属 性	说 明
closed	判断文件是否关闭，如果文件关闭，则返回 True
mode	返回文件的打开模式
name	返回文件的名称



文件对象的常用方法

方 法	说 明
<code>close()</code>	关闭文件，释放文件对象。
<code>flush()</code>	对于写模式，把缓冲区的内容写入文件，但不关闭文件
<code>read([size])</code>	从文本文件中读取 <code>size</code> 个字符的内容返回，或从二进制文件中读取 <code>size</code> 个字节的内容返回，如果省略 <code>size</code> 则表示读取所有内容
<code>readline()</code>	从文件中读取一行内容返回
<code>readlines()</code>	把文件中的每一行都作为一个字符串存入列表中，返回该列表
<code>seek(offset[, whence])</code>	把文件指针移动到新的字节位置， <code>offset</code> 表示相对于 <code>whence</code> 的位置。 <code>whence</code> 为 0 表示从文件头开始计算，1 表示从当前位置开始计算，2 表示从文件末尾开始计算，默认为 0
<code>tell()</code>	返回文件指针的当前位置（字节数）
<code>write(str)</code>	把 <code>str</code> 内容写入文件
<code>writelines(s_list)</code>	把字符串列表 <code>s_list</code> 写入文本文件，不添加换行符



文件对象



- 文件操作结束后需要调用`close()`方法关闭该文件。
- 在实际开发中，读写文件通常使用上下文管理语句**with**。

```
with open("input.txt") as f:  
    # 文件操作
```

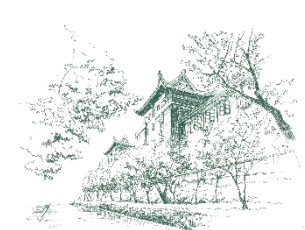
- **with**语句可以自动管理资源，不论何种原因跳出**with**块，总能保证文件被正确关闭。
- **with**语句常用于文件操作、数据库连接、网络连接、多线程与多进程同步时的锁对象管理等场合。



武汉大学
WUHAN UNIVERSITY

03

文本文件



文本文件的读取和写入

【例】创建文本文件sample.txt，写入一些内容，然后读取文件中的内容并输出。

```
s = "Hi\n文本文件的写入方法\n文本文件的读取方法\n"
with open("sample.txt", 'w') as f:
    f.write(s)

with open("sample.txt") as f:
    print(f.read())
```

程序运行后，在当前路径下生成sample.txt文件，同时得到下面的输出：

```
Hi
文本文件的写入方法
文本文件的读取方法
```




从文本文件中提取单词



【例】有一个文本文件about_python.txt，内容是关于Python的介绍。编写Python程序，读取文件内容，提取所有单词，并存放于列表中。

```
word_list = []
with open("about_python.txt") as f:
    for line in f: # 文件对象是可迭代对象
        lst = line.split()
        for word in lst:
            word_list.append(word.rstrip(",. ;?!")) # 删除字符串末尾的逗号、点号、分号、问号和感叹号
print(word_list)
```

```
['Python', 'is', 'powerful', 'and', 'fast', 'plays', 'well', 'with', 'others',
'runs', 'everywhere', 'is', 'friendly', '&', 'easy', 'to', 'learn', 'is', 'Open',
'These', 'are', 'some', 'of', 'the', 'reasons', 'people', 'who', 'use', 'Python',
'would', 'rather', 'not', 'use', 'anything', 'else']
```



武汉大学
WUHAN UNIVERSITY

04

CSV文件



CSV文件



■ CSV (Comma-Separated Values) 文件是一种常见的文件格式，用于存储和交换表格数据。它以纯文本形式存储数据，每行代表一条记录，字段之间用逗号分隔。

```
student_info.csv
文件 编辑 查看
学号,姓名,性别,专业,身高
301610163,王梦龙,男,测绘工程,175.5
302010260,李梓嘉,男,应用经济学,172.5
301060127,周迪,男,生物学,168.6
301000131,吴越,女,哲学,163.4
302011127,张昆仑,男,应用数学,170.9
301120020,金果果,女,环境工程,153.8
行 1, 列 1 | 174 个字符 | 100% | Windows (CR) | UTF-8
```

字符编码



CSV文件



■ Python标准库中的csv模块提供了读取和写入CSV文件的对象。

- ✿ csv.reader, 用于读取CSV文件, 并将每一行解析为一个列表
- ✿ csv.writer, 用于写入CSV文件, 将数据以列表的形式写入每一行
- ✿ csv.DictReader, 用于读取CSV文件, 并将每一行解析为一个字典
- ✿ csv.DictWriter, 用于写入CSV文件, 将字典形式的数据写入每一行, 字段名由指定的表头决定



CSV文件的读取

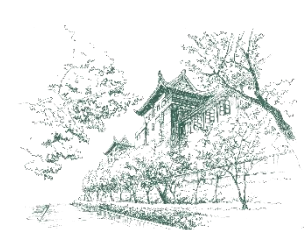


【例】通过csv.reader对象读取student_info.csv文件中的数据表。

```
import csv

with open("student_info.csv", encoding='utf-8', newline='') as f:
    csv_reader = csv.reader(f)    # 读取后返回一个csv.reader对象，利用该对象可以遍历csv文件
    for row in csv_reader:
        print(row)    # 遍历出来的row是列表
```

```
['学号', '姓名', '性别', '专业', '身高']
['301610163', '王梦龙', '男', '测绘工程', '175.5']
['302010260', '李梓嘉', '男', '应用经济学', '172.5']
['301060127', '周迪', '男', '生物学', '168.6']
['301000131', '吴越', '女', '哲学', '163.4']
['302011127', '张昆仑', '男', '应用数学', '170.9']
['301120020', '金果果', '女', '环境工程', '153.8']
```



CSV文件的读取



【例】通过csv.reader对象读取student_info.csv文件中的学生信息，并计算学生的平均身高。

```
import csv

with open("student_info.csv", encoding='utf-8', newline='') as f:
    csv_reader = csv.reader(f)
    next(csv_reader)      # 读取首行（标题）
    height = []
    for row in csv_reader:
        height.append(float(row[4])) # 读取的字段均为str类型
average = round(sum(height) / len(height), 2) # 保留小数点后2位
print(f"平均身高为: {average}")
```

平均身高为: 167.45



CSV文件的读取



【例】通过csv.DictReader对象读取student_info.csv文件中的数据表。

```
import csv

with open("student_info.csv", encoding='utf-8', newline='') as f:
    csv_dict_reader = csv.DictReader(f)    # 读取后返回一个csv.DictReader对象，利用该对象可以遍历csv文件
    for row in csv_dict_reader:
        print(row)    # 遍历出来的row是字典
```

```
{ '学号': '301610163', '姓名': '王梦龙', '性别': '男', '专业': '测绘工程', '身高': '175.5' }
{ '学号': '302010260', '姓名': '李梓嘉', '性别': '男', '专业': '应用经济学', '身高': '172.5' }
{ '学号': '301060127', '姓名': '周迪', '性别': '男', '专业': '生物学', '身高': '168.6' }
{ '学号': '301000131', '姓名': '吴越', '性别': '女', '专业': '哲学', '身高': '163.4' }
{ '学号': '302011127', '姓名': '张昆仑', '性别': '男', '专业': '应用数学', '身高': '170.9' }
{ '学号': '301120020', '姓名': '金果果', '性别': '女', '专业': '环境工程', '身高': '153.8' }
```



CSV文件的写入

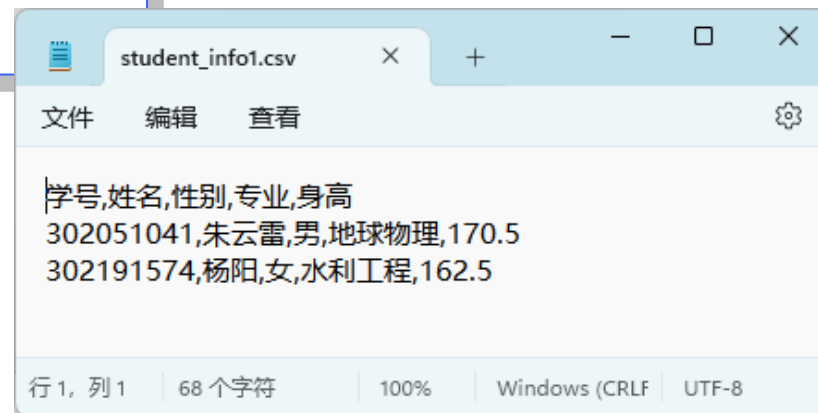


【例】创建student_info1.csv文件，通过csv.writer对象写入两名学生的基本信息数据。

```
import csv

headers = ['学号', '姓名', '性别', '专业', '身高']
rows = [('302051041', '朱云雷', '男', '地球物理', '170.5'),
        ('302191574', '杨阳', '女', '水利工程', '162.5')]
with open("student_info1.csv", 'w', newline='') as f:
    csv_writer = csv.writer(f)
    csv_writer.writerow(headers)  # 写入首行（每一列的标题）
    csv_writer.writerows(rows)   # 写入多行
```

程序运行后，在当前路径下生成student_info1.csv文件。





CSV文件的写入



【例】创建student_info2.csv文件，通过csv.DictWriter对象写入两名学生的基本信息数据。

```
import csv

headers = ['学号', '姓名', '性别', '专业', '身高']
rows = [{ '学号': '302051041', '姓名': '朱云雷', '性别': '男', '专业': '地球物理', '身高': '170.5' },
        { '学号': '302191574', '姓名': '杨阳', '性别': '女', '专业': '水利工程', '身高': '162.5' }]
with open("student_info2.csv", 'w', encoding='gb2312', newline='') as f:
    csv_dict_writer = csv.DictWriter(f, headers)
    csv_dict_writer.writeheader()    # 写入首行（每一列的标题）
    csv_dict_writer.writerows(rows) # 写入多行
```

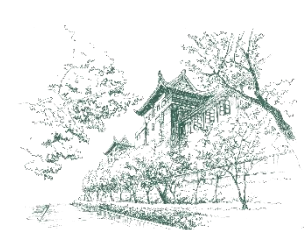
程序运行后，在当前路径下生成student_info2.csv文件，该文件内容同student_info1.csv



武汉大学
WUHAN UNIVERSITY

05

对象序列化



- 序列化是将Python对象转换为易于存储或传输的字节流或字符串表示的过程。
- 反序列化是将序列化数据转换回Python对象。
- Python的实现方法
 - ✱ 使用pickle模块中提供的函数，可以实现Python对象的序列化。
 - `pickle.dumps()`、`pickle.dump()`、`pickle.loads()`和`pickle.load()`
 - ✱ 推荐使用json模块中提供的函数，把Python对象转换为JSON字符串。
 - `json.dumps()`、`json.dump()`、`json.loads()`和`json.load()`



【例】 pickle.dump()函数的使用。

```
import pickle

data = {'a': [1, 2.0, 3 + 4j],
        'b': ("character string", b"byte string"),
        'c': {None, True, False} }
with open('data.pickle', 'wb') as f:
    pickle.dump(data, f, pickle.HIGHEST_PROTOCOL) # pickle.HIGHEST_PROTOCOL常量表示可用的最高协议版本
```

程序运行后，在当前路径下生成二进制文件**data.pickle**，其中存储了**data**对象的序列化结果。



pickle模块



【例】 从上例中生成的data.pickle文件中读取序列化数据，然后利用pickle.load()函数将数据反序列化为Python对象。

```
import pickle

with open('data.pickle', 'rb') as f:
    data = pickle.load(f)
print(data)
```

```
{ 'a': [1, 2.0, (3+4j)], 'b': ('character string', b'byte string'), 'c': {None, True, False}}
```



json模块和JSON格式数据



JSON

- ✿ JavaScript Object Notation, JavaScript对象标记。
- ✿ 定义了一种标准格式，用字符串来描述典型的内置对象（如字典、列表、数字和字符串）。
 - 虽然JSON原来是JavaScript编程语言的一个子集，但它现在是一个独立于语言的数据格式，所有主流编程语言都有生产和消费JSON数据的库。
- ✿ JSON是网络数据交换的流行格式之一。



json模块和JSON格式数据



【例】json.dumps()函数的使用。

```
import json

data = {'e': True, 'a': 3.14, 'c': 3, 'd': (2, 4), 'b': 'K'}
json_data = json.dumps(data, indent=4, sort_keys=True) # json_data为str类型
print(json_data)
```

```
{
    "a": 3.14,
    "b": "K",
    "c": 3,
    "d": [
        2,
        4
    ],
    "e": true
}
```



json模块和JSON格式数据



【例】json.dump()函数的使用。

```
import json

urls = { '百度': 'www.baidu.com',
        'bilibili': 'www.bilibili.com',
        '腾讯': 'www.tencent.com' }

with open("data.json", 'w') as f:
    json.dump(urls, f, ensure_ascii=False, indent=4)
```

程序运行后，在当前路径下生成data.json文件，该文件内容如下：





json模块和JSON格式数据



【例】json.loads()函数的使用。

```
import json

json_data = '{"a": 3.14, "b": "K", "c": 3, "d": [2, 4], "e": true}'
data = json.loads(json_data)    # python_data为dict类型
print(data)
```

```
{'a': 3.14, 'b': 'K', 'c': 3, 'd': [2, 4], 'e': True}
```



json模块和JSON格式数据



【例】从前例中生成的data.json文件中读取序列化数据，然后利用json.load()函数将数据反序列化为Python对象。

```
import json

with open("data.json") as f:
    urls = json.load(f)      # urls为dict类型
print(urls)
```

```
{'百度': 'www.baidu.com', 'bilibili': 'www.bilibili.com', '腾讯': 'www.tencent.com'}
```



json模块和JSON格式数据



■ 注意: json不支持集合对象的JSON序列化。

```
import json

courses = {'数据科学导论', 'Python', '数据分析'}

with open('file5.json', 'w') as f:
    json.dump(courses, f)
```

TypeError: Object of type set is not JSON serializable



武汉大学
WUHAN UNIVERSITY

06

异常处理



异常处理



武汉大学
WUHAN UNIVERSITY

■ 异常是指程序运行时引发的错误

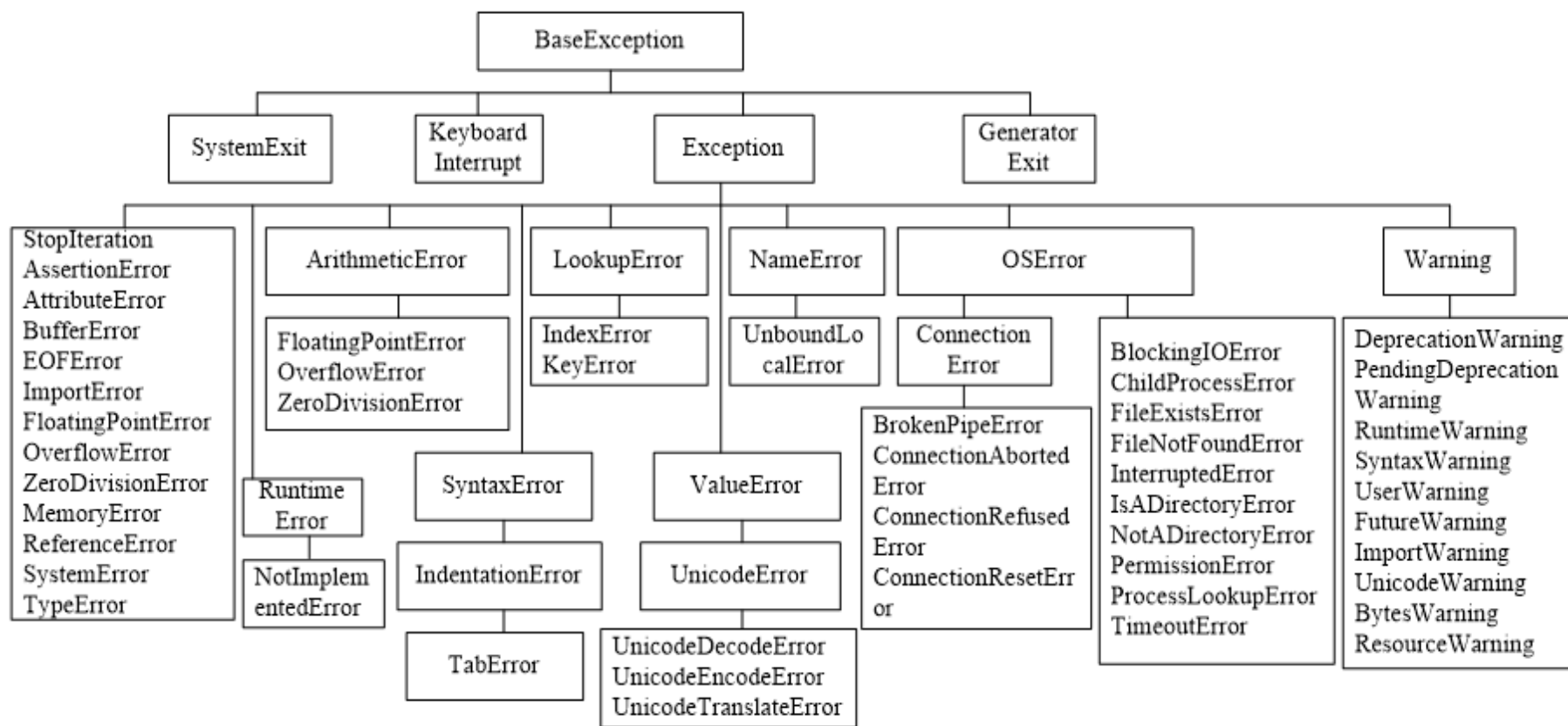
✱ 引发错误的原因有很多，例如除零、下标越界、文件不存在、网络异常、类型错误、名字错误、字典键错误、磁盘空间不足……，如果这些错误得不到正确的处理将会导致程序终止运行。

■ 合理地使用异常处理结构可以使程序更加健壮，具有更强的容错性，不会因为用户不小心的错误输入或其他运行时原因而造成程序崩溃或产生不可预期的结果。



■ 内置异常类

在Python中，所有异常都是一个派生自BaseException的类的实例。





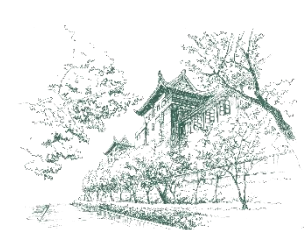
■ 异常的发生

✿ 当程序在执行过程中出现错误时Python会自动引发异常

```
123 + 'abc'    # TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

✿ 用户也可以创建相应异常类的对象，并通过raise语句抛出

```
a = int(input("请输入一个整数: "))  
if a < 0: raise ValueError("数值不能为负数")  
print(a ** 0.5)
```



■ 异常的捕捉和处理

✿ Python采用结构化的异常处理机制。

- **try**子句定义代码块，运行可能抛出异常的代码；
- **except**子句捕获特定的异常并执行相应的处理；
- **else**子句执行无异常时的处理；
- **finally**子句保证即使产生异常（处理失败），也可以在事后清理资源等。



■ 异常的捕捉和处理

✿ 完整的try...except...else...finally语句一般格式

```
try:
    可能产生异常的语句
except Exception1:
    发生异常时执行的语句
except (Exception2, Exception3):
    发生异常时执行的语句
except Exception4 as e:
    发生异常时执行的语句
except:
    发生异常时执行的语句
else:
    无异常时执行的语句
finally:
    不管发生异常与否都保证执行的语句
```

捕获异常 Exception1

捕获异常 Exception2、Exception3

捕获异常 Exception4,其实例为 e

捕获其他所有异常

无异常

不管发生异常与否都保证执行



【例】打开文件testfile.txt，写入一些内容，同时捕获可能发生的异常。

```
try:
    f = open("testfile.txt", "w")
    f.write("这是一个测试文件，用于测试异常!!")
    f1 = open("testfile1.txt", "r")
except OSError:
    print("没有找到文件或读取文件失败")
else:
    print("文件写入成功！")
finally:
    f.close()
```

如果testfile.txt写入成功，但testfile1.txt不存在，则程序得到下面的输出：

没有找到文件或读取文件失败

如果testfile.txt和testfile1.txt都操作成功，则程序得到下面的输出：

文件写入成功！



■ 异常的捕捉和处理

- ✱ 当有多个except子句时，一旦try子句中的代码抛出了异常，就按顺序依次检查与哪一个except子句匹配，如果某个except捕捉到了异常，其他的except子句将不会再尝试捕捉异常。

```
lst = [-15, 66, 79, -7, 2]
total = 0
try:
    for i in lst:
        if i < 0: raise ValueError(str(i) + "为负数")
        total += i
    print("合计 =", total)
except Exception:
    print("发生异常")
except ValueError: #派生程度高的异常类ValueError放置在派生程度低的Exception后面，导致无法捕获
    print("数值不能为负")
```

发生异常



异常处理



【例】 函数div()接收两个参数x和y，输出x除以y的结果，根据不同的情况进行相应的异常处理。

```
def div(x, y):  
    try:  
        print(x / y)  
    except ZeroDivisionError:  
        print('ZeroDivisionError')  
    except TypeError:  
        print('TypeError')  
    else:  
        print('No Error')  
    finally:  
        print("Executing finally clause")
```

```
>>> div(3, 5)  
0.6  
No Error  
Executing finally clause  
>>> div('3', 5)  
TypeError  
Executing finally clause  
>>> div(3, 0)  
ZeroDivisionError  
Executing finally clause
```



异常处理



【例】将输入的字符串写入到文本中，直至按Q结束。如果按Ctrl+C中断程序运行，也保证打开的文件正常关闭。

```
try:
    f = open('mytext.txt', 'w') # 打开要写入的文件
    while True:
        s = input('请输入字符串（按Q结束）： ')
        if s.upper() == 'Q': break
        f.write(s + '\n')
except KeyboardInterrupt:
    print('程序中断！（Ctrl-C）')
finally:
    f.close()
```

请输入字符串（按Q结束）： e
请输入字符串（按Q结束）： q

请输入字符串（按Q结束）： 4
请输入字符串（按Q结束）： 程序中断！（Ctrl-C）



武汉大学
WUHAN UNIVERSITY

07

实践指导



数据采集



武汉大学
WUHAN UNIVERSITY

■ **数据采集**，又称**数据获取**，是通过多种技术手段把外部各种数据源产生的数据进行实时或非实时地采集并加以利用。

- ✱ **传感器采集**

- ✱ **网络数据采集**

 - 通过网络爬虫或网站公开**API**（应用程序编程接口）从万维网获取数据信息。

- ✱ **系统日志采集**

- ✱ **企业业务系统数据采集**



通过API采集特定领域的的数据

【例】通过百度API获取武汉市实时天气信息及未来5天天气预报数据。

✿ 步骤一：注册并获取百度地图开放平台的API密钥。

- (1) 打开百度地图开放平台（<https://lbsyun.baidu.com/>）；
- (2) 注册一个账号或登录已有账号；
- (3) 在控制台中创建一个新的应用，并申请天气相关服务的API权限。
- (4) 获取密钥（AK和SK），密钥将用于身份验证。



通过API采集特定领域的的数据

【例】通过百度API获取武汉市实时天气信息及未来5天天气预报数据。

✿ 步骤二：了解国内天气查询接口文档。

百度天气查询服务是一套REST风格的Web服务API，以HTTP形式提供了实时和未来天气查询服务。以下是接口文档中的关键信息：

(1) API服务地址：

https://api.map.baidu.com/weather/v1/?district_id=区县的行政区划编码&data_type=请求数据类型&ak=开发者密钥

(2) 请求方式：GET

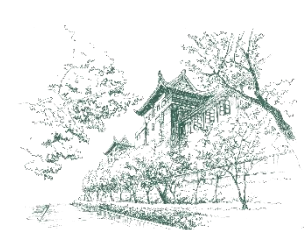
(3) 部分参数：

district_id: 区县的行政区划编码；

ak: 开发者密钥（AK）；

data_type: 请求数据类型；

✿ 步骤三：根据接口文档中的示例，编写Python程序。



通过API采集特定领域的的数据



武汉大学
WUHAN UNIVERSITY

【例】百度API获取武汉市实时天气信息及未来5天天气预报数据。

```
import requests
import urllib
import hashlib
import json
# 服务主机地址
host = "https://api.map.baidu.com"
# 接口地址
uri = "/weather/v1/"
# 你的应用密钥AK
ak = ""
# 在控制台创建应用时，校验方式选择sn校验后生成的SK
sk = ""
# 设置请求参数
params = {
    "district_id": "420100",      # 武汉市的行政区划编码
    "data_type": "all",
    "ak": ak
}
# 拼接请求字符串
paramsArr = []
for key in params:
    paramsArr.append(key + "=" + params[key])
queryStr = uri + "?" + "&".join(paramsArr)
```



通过API采集特定领域的的数据



武汉大学
WUHAN UNIVERSITY

【例】百度API获取武汉市实时天气信息及未来5天天气预报数据。

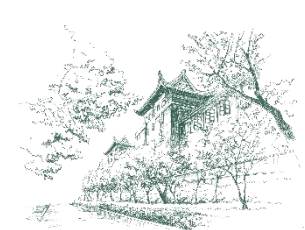
```
# 对queryStr进行转码, safe内的保留字符不转换
encodedStr = urllib.request.quote(queryStr, safe="/:=&?#+!$,;'@()*[]")

# 在最后直接追加你的sk
rawStr = encodedStr + sk

# 计算sn
sn = hashlib.md5(urllib.parse.quote_plus(rawStr).encode("utf8")).hexdigest()

# 将sn参数添加到请求中
queryStr = queryStr + "&sn=" + sn

url = host + queryStr
response = requests.get(url)
if response:
    with open("weather_data.json", 'w') as f:
        json.dump(response.json(), f, ensure_ascii=False, indent=4)
else:
    print("请求失败, 状态码: ", response.status_code)
```



小结



- 把数据写入文件是实现数据长期存储的基本方法，这将涉及到如何创建文件，把数据写入文件，然后从文件中正确读出数据。
- Python的标准库中有大量支持文件操作的模块，用以实现文本文件、CSV文件的读写操作。
- 对象序列化可以把对象保存到文件，然后从文件中正确读出，而无需做类型转换。
- 在编写函数或类时，有必要对其进行测试，通过测试确定代码面对各种输入都能够按要求工作。
- 为了增强程序的健壮性，有必要对异常进行检测和处理。
- 根据数据的不同来源，数据采集包括传感器采集、网络数据采集和日志文件采集和企业业务系统采集等。



武汉大学

WUHAN UNIVERSITY

谢谢

THANK YOU

